

Memoria Práctica 3 Sistemas Operativos

Carlos Méndez Barrasa y Ana Olsson Rodríguez

Grupo

2262

Índice

1. Requisitos cumplidos y no cumplidos.....	3
1.1. Sistema de monitorización.....	3
1.2. Sustitución de ficheros por memoria compartida.....	3
1.3. Productor-consumidor entre Comprobador y Monitor.....	3
2. Pruebas realizadas.....	4
2.1. Ejecución en varias terminales.....	4
2.1.1. En VSCode.....	4
2.1.1. En WSL.....	5
2.2. Liberación de recursos y contraste con logs.....	6
2.3. Ejecución de un minero sin monitor previo.....	7
2.4. Retardos desiguales entre monitor y comprobador.....	8
2.4.1. Retardo mayor en monitor.....	8
2.4.1. Retardo mayor en comprobador.....	9
2.5. Parámetros no válidos para monitor.....	10
2.6. Valgrind.....	10
3. Cuestiones.....	13
3.1. ¿Es necesario un sistema productor-consumidor si el retraso de Comprobador es mucho mayor que el de Minero?.....	13
3.2. ¿Y si el retraso de Comprobador es muy inferior al de Minero?.....	13
3.3. ¿Se simplificaría la estructura global si entre Comprobador y Monitor hubiera también una cola de mensajes?.....	13

1. Requisitos cumplidos y no cumplidos

En esta práctica se amplían las prácticas anteriores de Miner Rush incorporando un mecanismo de monitorización centralizado, comunicación mediante colas de mensajes POSIX y sustitución de estructuras temporales basadas en fichero por memoria compartida.

1.1. Sistema de monitorización

Se implementa un programa monitor que recibe dos parámetros, el retardo del `Comprobador` y el retardo del `Monitor`. El programa crea dos procesos mediante `fork`: el padre actúa como `Comprobador` y el hijo como `Monitor`.

El proceso `Monitor` es el encargado de crear el segmento de memoria compartida, inicializarlo, crear el semáforo nominal usado por los mineros y preparar los tres semáforos anónimos del productor-consumidor. El proceso `Comprobador` abre la cola de mensajes, recibe bloques enviados por los mineros, valida la solución y deposita el resultado en el búfer circular compartido para que el `Monitor` lo imprima.

También se observa la terminación ordenada de recursos: el `Comprobador` cierra y elimina la cola de mensajes, mientras que el `Monitor` destruye los semáforos anónimos, desmonta la memoria compartida y realiza `shm_unlink`. Además, si un minero arranca antes de que exista la cola, falla la apertura con `mq_open` y termina con error.

1.2. Sustitución de ficheros por memoria compartida

La memoria compartida se usa para almacenar el estado global del sistema: número de mineros, participantes de una ronda, votos, objetivo actual, ganador, solución ganadora, carteras y el búfer circular empleado entre `Comprobador` y `Monitor`. Se ha reemplazado por memoria compartida las estructuras comunes que en prácticas anteriores se guardaban en ficheros.

1.3. Productor-consumidor entre Comprobador y Monitor

La comunicación entre `Comprobador` y `Monitor` se implementa mediante un búfer circular de 6 bloques en memoria compartida, protegido con tres semáforos anónimos: `sem_empty`, `sem_fill` y `sem_mutex`. El `Comprobador` hace de productor, insertando bloques validados, y el `Monitor` hace de consumidor, extrayéndolos e imprimiéndolos con el formato pedido para soluciones aceptadas y rechazadas.

Mineros:

```

$ ./miner 10 8000
Miner 3678564 added to system
Current miners: 3678564

--- Initial target created: 0 ---

[3678564] Generating blocks...
Winner 3678564
Winner 3678564
Winner 3678564
Winner 3678564
Winner 3678564
Winner 3678564
Winner 3678564
[3678564] Finishing (Miner)
Logger exited with status 0
Current miners: 3720398 3678597

$ ./miner 15 8000
Miner 3678597 added to system
Current miners: 3678564 3678597
[3678597] Generating blocks...
Winner 3678597
Winner 3678597
Winner 3678597
Winner 3678597
Winner 3678597
Winner 3678597
Winner 3678597
[3678597] Finishing (Miner)
Logger exited with status 0
Current miners: 3720398

$ ./miner 20 8000
Miner 3720398 added to system
Current miners: 3678564 3678597 3720398
[3720398] Generating blocks...
Winner 3720398
Winner 3720398
[3720398] Finishing (Miner)
Logger exited with status 0

--- X No miners left ---

```

2.1.1. En WSL

```

T1 $ ./monitor 10 18
Welcome to Miner Rush!
Shared memory segment open
[505665] Printing blocks...
[505664] Checking blocks...
Miner 505679 was broke, gifting them a wallet as well, they now have 1 coin
Solution accepted: 000000000 -> 00916551
Miner 505679 was broke, gifting them a wallet as well, they now have 1 coin
Solution accepted: 00916551 -> 05970796
Miner 505679 was given their coin, they now have 2 coins
Solution accepted: 05970796 -> 00937352
Miner 521681 was broke, gifting them a wallet as well, they now have 1 coin
Solution accepted: 00937352 -> 01835314
Miner 505679 was given their coin, they now have 3 coins
Solution accepted: 01835314 -> 09332340
Miner 521681 was given their coin, they now have 2 coins
Solution accepted: 09332340 -> 02906686
[505664] Finishing (Comproador)
[505665] Finishing (Monitor)
The Miner Rush has ended

T2 $ ./miner 2 8000
Miner 505671 added to system
Current miners: 505671

--- Initial target created: 0 ---

[505671] Generating blocks...
Winner 505671
[505671] Finishing (Miner)
Logger exited with status 0
Current miners: 521681 505679
T2 $

T3 $ ./miner 5 8000
Miner 505679 added to system
Current miners: 505671 505679
[505679] Generating blocks...
Winner 505679
Miner 505679
[505679] Finishing (Miner)
Logger exited with status 0
Current miners: 521681
T3 $

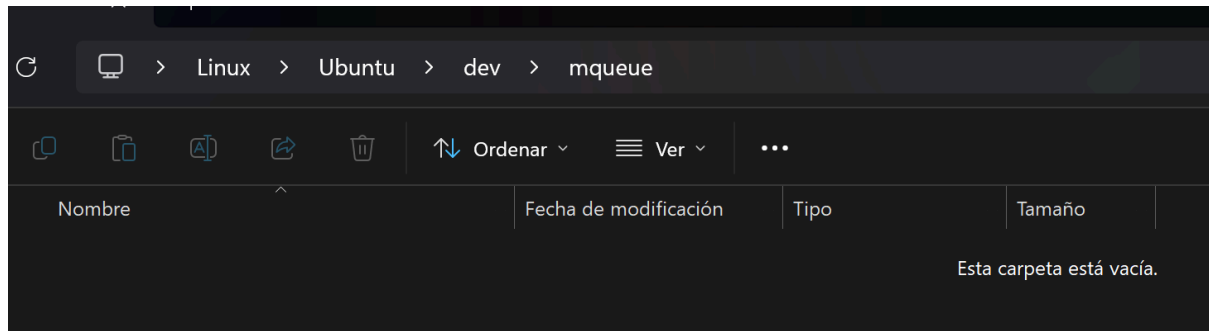
T4 $ ./miner 7 8000
Miner 521681 added to system
Current miners: 505671 505679 521681
[521681] Generating blocks...
Winner 521681
[521681] Finishing (Miner)
Logger exited with status 0
--- X No miners left ---
T4 $

```

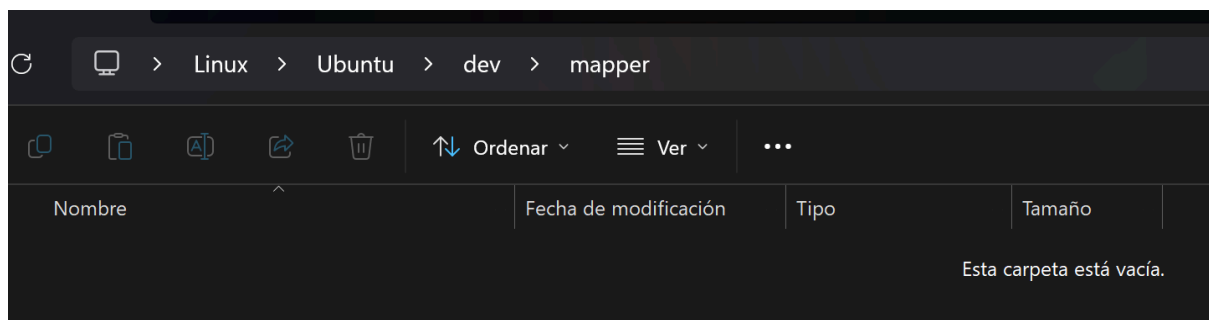
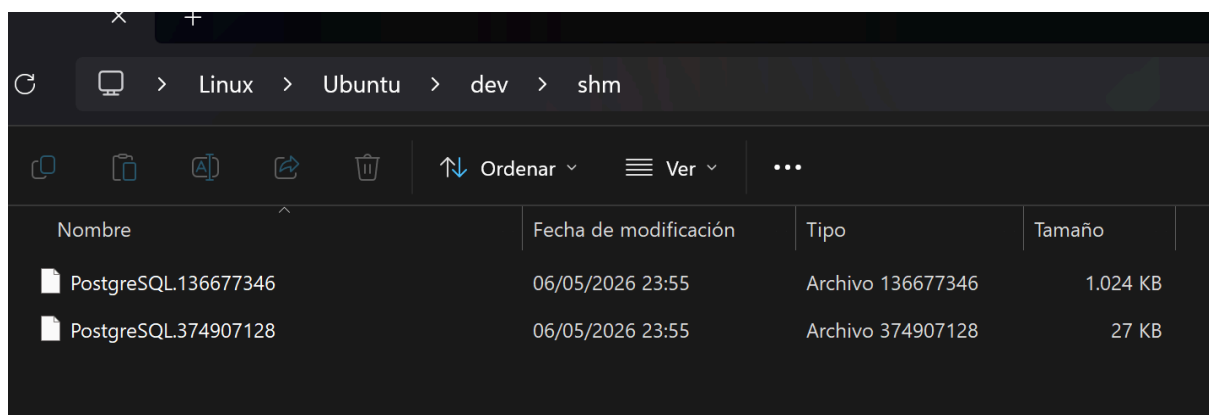
Como podemos apreciar en las imágenes, el programa espera y reconoce a los mineros permitiéndoles participar y ganar rondas, se intentó que los mineros se lanzaran a la vez para realizar el mayor número de rondas posible cambiando entre terminales para ejecutarlos seguidamente. Para recabar más información de las rondas podemos observar el monitor ya que posee diversa impresión de información por pantalla, los colores se han puesto para mejorar la legibilidad.

Además, si consultamos las carpetas del sistema de Linux tras la ejecución del programa vemos que no se han quedado recursos sin liberar (ya sean colas de mensajes, semáforos o memoria compartida):

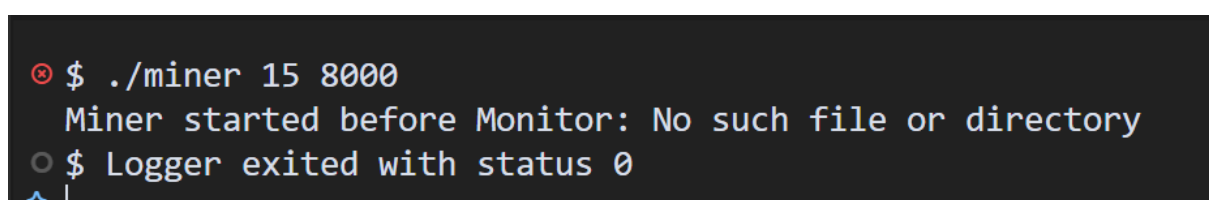
Colas de mensajes:



Semáforos y Memoria compartida:



2.3. Ejecución de un minero sin monitor previo



2.4. Retardos desiguales entre monitor y comprobador

2.1.1. Retardo mayor en monitor

[illegible]

Como podemos observar el programa se ejecuta sin fallo aunque los prints se descuadran debido a que los procesos se están ejecutando en una misma terminal y el lag retiene los de un proceso.

2.1.1. Retardo mayor en comprobador

[illegible]

Aquí también se ejecuta el programa sin errores aunque los prints se descuadran debido a que los procesos se están ejecutando en una misma terminal.

2.5. Parámetros no válidos para monitor

```

❌ $ ./monitor -10 10
Invalid arguments
: Success
❌ $ ./monitor 10 -10
Invalid arguments
: Success
❌ $ ./monitor -10 -10
Invalid arguments
: Success

```

2.6. Valgrind

Ejecución:

```

$ valgrind --leak-check=full --track-origins=yes --trace-children=yes --log-file="valgrind_monitor.log" ./monitor 10 10
===== Welcome to Miner Rush! =====
Shared memory segment created
[2814880] Printing blocks...
[2814702] Checking blocks...
Miner 2814839 was broke, gifting them a wallet as well, they now have 1 coin
Solution accepted: 00000000 --> 08916551
Miner 2814881 was broke, gifting them a wallet as well, they now have 1 coin
Solution accepted: 08916551 --> 05970796
Miner 2814881 was given their coin, they now have 2 coins
Solution accepted: 05970796 --> 00937352
Miner 2814839 was given their coin, they now have 2 coins
Solution accepted: 00937352 --> 01835314
Miner 2814886 was broke, gifting them a wallet as well, they now have 1 coin
Solution accepted: 01835314 --> 09331340
Miner 2814886 was given their coin, they now have 2 coins
Solution accepted: 09331340 --> 02986686
Miner 2814886 was given their coin, they now have 3 coins
Solution accepted: 02986686 --> 07299854
Miner 2814839 was given their coin, they now have 3 coins
Solution accepted: 07299854 --> 03981310
Miner 2814839 was given their coin, they now have 4 coins
Solution accepted: 03981310 --> 08021305
Miner 2814886 was given their coin, they now have 4 coins
Solution accepted: 08021305 --> 04673876
[2814762] Finishing (Comprobador)
[2814880] Finishing (Monitor)
===== The Miner Rush has ended =====

$ valgrind --leak-check=full --track-origins=yes --trace-children=yes --log-file="valgrind_miner1.log" ./miner 5 5
Miner 2814801 added to system
Current miners: 2814801
--- Initial target created: 0 ---
[2814801] Generating blocks...
Winner 2814801
[2814801] Finishing (Miner)
Logger exited with status 0
Current miners: 2814886 2814839
$

$ valgrind --leak-check=full --track-origins=yes --trace-children=yes --log-file="valgrind_miner1.log" ./miner 10 2
Miner 2814839 added to system
Current miners: 2814880 2814839
[2814839] Generating blocks...
Winner 2814839
Miner 2814839
Miner 2814839
Miner 2814839
[2814839] Finishing (Miner)
Logger exited with status 0
Current miners: 2814886
$

$ valgrind --leak-check=full --track-origins=yes --trace-children=yes --log-file="valgrind_miner1.log" ./miner 15 5
Miner 2814886 added to system
Current miners: 2814880 2814839 2814886
[2814886] Generating blocks...
Winner 2814886
Miner 2814886
Miner 2814886
[2814886] Finishing (Miner)
Logger exited with status 0
--- X no miners Left ---
$

```

Logs generados:

```

≡ valgrind_monitor.log U X
Practica_3 > Ejercicio_7 > ≡ valgrind_monitor.log
1  ==2814762== Memcheck, a memory error detector
2  ==2814762== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
3  ==2814762== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
4  ==2814762== Command: ./monitor 10 10
5  ==2814762== Parent PID: 2814079
6  ==2814762==
7  ==2814800==
8  ==2814800== HEAP SUMMARY:
9  ==2814800==      in use at exit: 0 bytes in 0 blocks
10 ==2814800==    total heap usage: 3 allocs, 3 frees, 1,091 bytes allocated
11 ==2814800==
12 ==2814800== All heap blocks were freed -- no leaks are possible
13 ==2814800==
14 ==2814800== For lists of detected and suppressed errors, rerun with: -s
15 ==2814800== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
16 ==2814762==
17 ==2814762== HEAP SUMMARY:
18 ==2814762==      in use at exit: 67 bytes in 2 blocks
19 ==2814762==    total heap usage: 3 allocs, 1 frees, 1,091 bytes allocated
20 ==2814762==
21 ==2814762== LEAK SUMMARY:
22 ==2814762==      definitely lost: 0 bytes in 0 blocks
23 ==2814762==      indirectly lost: 0 bytes in 0 blocks
24 ==2814762==      possibly lost: 0 bytes in 0 blocks
25 ==2814762==      still reachable: 67 bytes in 2 blocks
26 ==2814762==      suppressed: 0 bytes in 0 blocks
27 ==2814762== Reachable blocks (those to which a pointer was found) are not shown.
28 ==2814762== To see them, rerun with: --leak-check=full --show-leak-kinds=all
29 ==2814762==
30 ==2814762== For lists of detected and suppressed errors, rerun with: -s
31 ==2814762== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
32

```

```

≡ valgrind_miner1.log U X
Practica_3 > Ejercicio_7 > ≡ valgrind_miner1.log
11 ==2814917== All heap blocks were freed -- no leaks are possible
12 ==2814917==
13 ==2814917== For lists of detected and suppressed errors, rerun with: -s
14 ==2814917== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
15 ==2814886==
16 ==2814886== HEAP SUMMARY:
17 ==2814886==      in use at exit: 0 bytes in 0 blocks
18 ==2814886==    total heap usage: 169 allocs, 169 frees, 9,778 bytes allocated
19 ==2814886==
20 ==2814886== All heap blocks were freed -- no leaks are possible
21 ==2814886==
22 ==2814886== For lists of detected and suppressed errors, rerun with: -s
23 ==2814886== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
24

```

```

≡ valgrind_miner2.log U X
Practica_3 > Ejercicio_7 > ≡ valgrind_miner2.log
1  ==2813567== Memcheck, a memory error detector
2  ==2813567== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
3  ==2813567== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
4  ==2813567== Command: ./miner 10 5
5  ==2813567== Parent PID: 4417
6  ==2813567==
7  ==2813669==
8  ==2813669== HEAP SUMMARY:
9  ==2813669==      in use at exit: 0 bytes in 0 blocks
10 ==2813669==    total heap usage: 1 allocs, 1 frees, 1,024 bytes allocated
11 ==2813669==
12 ==2813669== All heap blocks were freed -- no leaks are possible
13 ==2813669==
14 ==2813669== For lists of detected and suppressed errors, rerun with: -s
15 ==2813669== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
16 ==2813567==
17 ==2813567== HEAP SUMMARY:
18 ==2813567==      in use at exit: 0 bytes in 0 blocks
19 ==2813567==    total heap usage: 69 allocs, 69 frees, 5,342 bytes allocated
20 ==2813567==
21 ==2813567== All heap blocks were freed -- no leaks are possible
22 ==2813567==
23 ==2813567== For lists of detected and suppressed errors, rerun with: -s
24 ==2813567== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
25

```

```

≡ valgrind_miner3.log U X
Practica_3 > Ejercicio_7 > ≡ valgrind_miner3.log
1  ==2813571== Memcheck, a memory error detector
2  ==2813571== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
3  ==2813571== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
4  ==2813571== Command: ./miner 15 5
5  ==2813571== Parent PID: 2813568
6  ==2813571==
7  ==2813671==
8  ==2813671== HEAP SUMMARY:
9  ==2813671==      in use at exit: 0 bytes in 0 blocks
10 ==2813671==    total heap usage: 1 allocs, 1 frees, 1,024 bytes allocated
11 ==2813671==
12 ==2813671== All heap blocks were freed -- no leaks are possible
13 ==2813671==
14 ==2813671== For lists of detected and suppressed errors, rerun with: -s
15 ==2813671== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
16 ==2813571==
17 ==2813571== HEAP SUMMARY:
18 ==2813571==      in use at exit: 0 bytes in 0 blocks
19 ==2813571==    total heap usage: 137 allocs, 137 frees, 8,163 bytes allocated
20 ==2813571==
21 ==2813571== All heap blocks were freed -- no leaks are possible
22 ==2813571==
23 ==2813571== For lists of detected and suppressed errors, rerun with: -s
24 ==2813571== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
25

```

3. Cuestiones

3.1. ¿Es necesario un sistema productor-consumidor si el retraso de Comprobador es mucho mayor que el de Minero?

Sí. Si los mineros pueden generar resultados más deprisa de lo que el **Comprobador** es capaz de validarlos o reenviarlos, aparece un desacoplamiento claro entre productor y consumidor. Sin un búfer intermedio protegido con semáforos, habría riesgo de sobreescritura de datos compartidos, pérdida de bloques o necesidad de bloqueo activo. El patrón productor-consumidor resuelve esto permitiendo almacenar temporalmente varios bloques hasta que el consumidor pueda procesarlos.

3.2. ¿Y si el retraso de Comprobador es muy inferior al de Minero?

También sigue teniendo sentido. En ese caso el búfer no suele llenarse, pero el esquema sigue garantizando corrección y sincronización. El **Comprobador** simplemente quedará bloqueado esperando nuevos elementos en `sem_fill`, qué es exactamente el comportamiento deseable. Es decir, cuando el consumidor es más rápido, el modelo no perjudica; sólo introduce una sincronización limpia y general.

3.3. ¿Se simplificaría la estructura global si entre Comprobador y Monitor hubiera también una cola de mensajes?

No necesariamente. Podría simplificar algunos detalles de sincronización manual, porque las colas de mensajes ya incorporan una semántica FIFO y bloqueo bien definida. Sin embargo, también supondría usar otro recurso del sistema operativo, duplicar la lógica de apertura/cierre/unlink y desaprovechar uno de los objetivos principales de la práctica, que es trabajar con memoria compartida y semáforos.